Experiment - 5

Nikhil Kale
D15A/50

Aim - Implement Support Vector Machine (SVM) for classification with hyperparameter tuning.

1. Dataset Source

Dataset used:
https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset

Title: IBM HR Analytics Employee Attrition Dataset

2. Dataset Description

The dataset contains employee information used to analyze and predict employee attrition.

Total records: **1470 employees**

Total features: **35 attributes**

Target Variable:

Attrition

- Yes → Employee left the company
- No → Employee stayed

Important Features:

- Age
- JobSatisfaction
- MonthlyIncome
- YearsAtCompany
- JobRole
- OverTime
- Education
- DistanceFromHome

Dataset Characteristics:

- Mixed categorical and numerical features

- No missing values
- Binary classification problem

3. Mathematical Formulation

Support Vector Machine finds the optimal hyperplane that separates classes.

Decision boundary:

$$w \cdot x + b = 0$$

Goal:

Maximize margin between two classes.

Optimization:

$$\min \frac{1}{2} ||w||^2$$

Subject to:

$$y_i(w \cdot x_i + b) \geq 1$$

For non-linear data, kernel functions are used.

Common kernels:

- Linear
- Polynomial
- Radial Basis Function (RBF)

4. Algorithm Limitations

- Computationally expensive for large datasets
- Sensitive to feature scaling
- Difficult to interpret

- Choosing kernel parameters is challenging
- Poor performance when classes overlap heavily

5. Methodology / Workflow

Steps followed in the experiment:

1. Load dataset
2. Perform exploratory data analysis
3. Encode categorical variables
4. Remove irrelevant columns
5. Train-test split
6. Feature scaling
7. Train SVM model
8. Hyperparameter tuning using GridSearchCV
9. Evaluate model performance

Workflow:

Dataset → Preprocessing → Feature Encoding → Scaling → Train Model → Hyperparameter Tuning → Evaluation

6. Performance Analysis

Evaluation metrics used:

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix

The tuned SVM model generally performs better because the optimal kernel and parameters improve classification boundaries.

7. Hyperparameter Tuning

Hyperparameters tuned:

- **C (Regularization parameter)**
- **Gamma**
- **Kernel type**

GridSearchCV with 5-fold cross validation was used to find the best parameter combination.

Proper tuning improves model generalization and classification performance.

Code and Output -

[1]
2s

```python
# Data Handling
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# ML libraries
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

plt.style.use("seaborn-v0_8")
```

[4]
0s

```python
df = pd.read_csv("/content/WA_Fn-UseC_-HR-Employee-Attrition.csv")

df.head()
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeNumber	...	Re
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life Sciences	1	1	...	
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life Sciences	1	2	...	
2	37	Yes	Travel_Rarely	1373	Research &	2	2	Other	1	4		

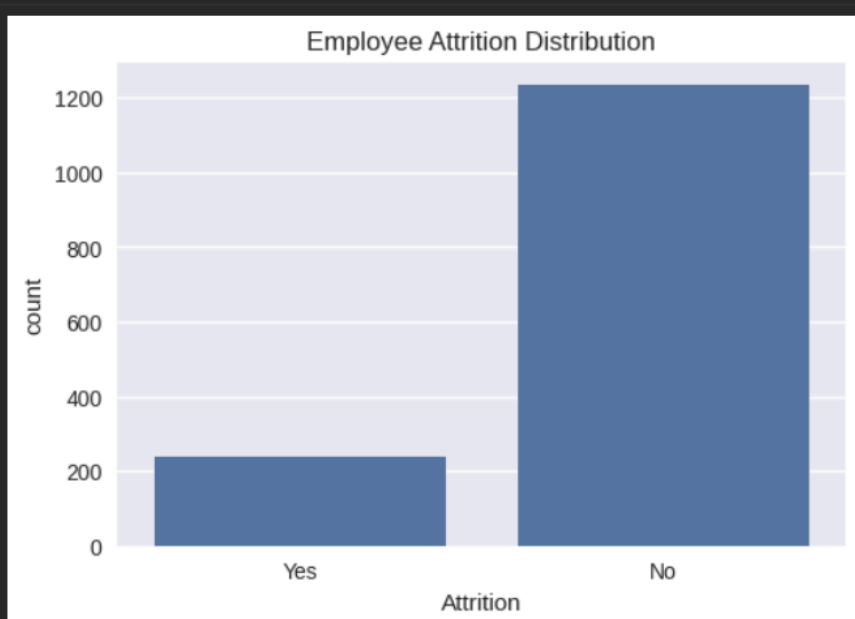
[5]
0s

```python
df.shape
df.info()
df.describe()
df.isnull().sum()
```

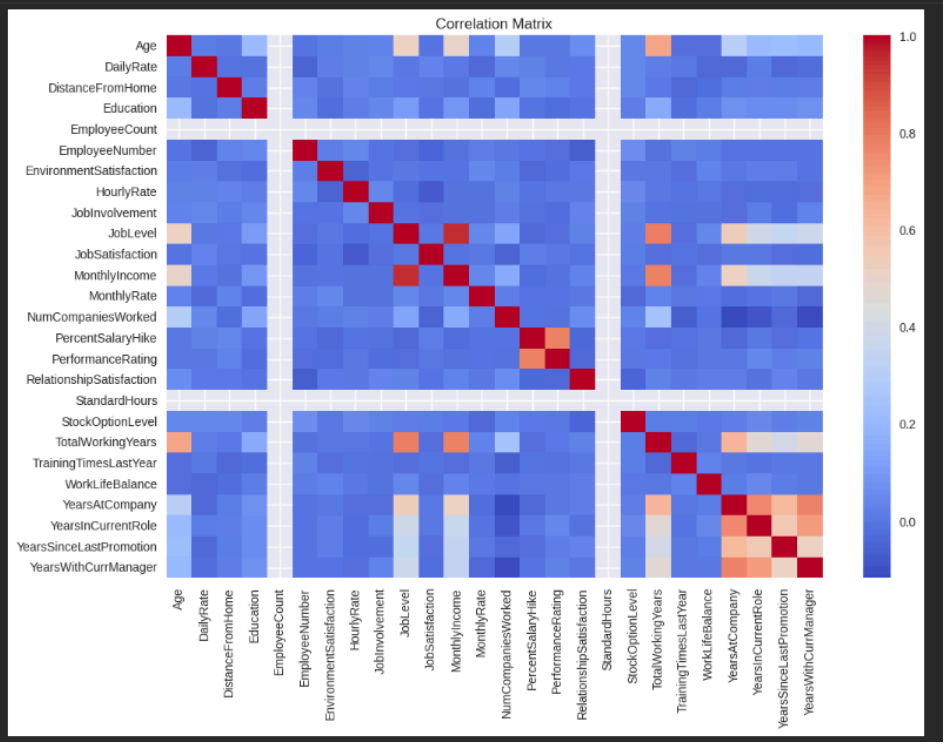
```
 12  HourlyRate                1470 non-null   int64
 13  JobInvolvement            1470 non-null   int64
 14  JobLevel                  1470 non-null   int64
 15  JobRole                   1470 non-null   object
 16  JobSatisfaction           1470 non-null   int64
 17  MaritalStatus             1470 non-null   object
 18  MonthlyIncome             1470 non-null   int64
 19  MonthlyRate               1470 non-null   int64
 20  NumCompaniesWorked        1470 non-null   int64
 21  Over18                    1470 non-null   object
 22  OverTime                  1470 non-null   object
 23  PercentSalaryHike         1470 non-null   int64
 24  PerformanceRating         1470 non-null   int64
 25  RelationshipSatisfaction  1470 non-null   int64
 26  StandardHours             1470 non-null   int64
 27  StockOptionLevel          1470 non-null   int64
 28  TotalWorkingYears         1470 non-null   int64
 29  TrainingTimesLastYear     1470 non-null   int64
 30  WorkLifeBalance           1470 non-null   int64
 31  YearsAtCompany            1470 non-null   int64
 32  YearsInCurrentRole        1470 non-null   int64
 33  YearsSinceLastPromotion   1470 non-null   int64
 34  YearsWithCurrManager      1470 non-null   int64
dtypes: int64(26), object(9)
memory usage: 402.1+ KB
```

[6]
0s

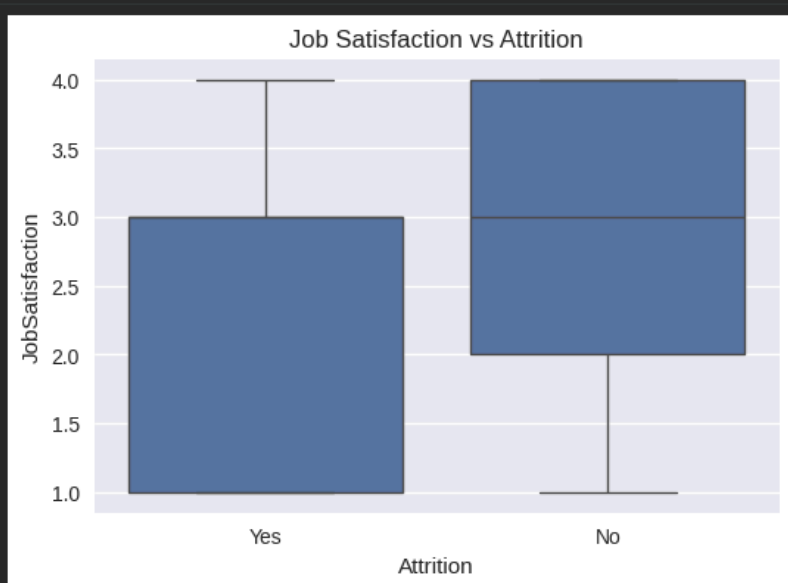
```python
plt.figure(figsize=(6,4))
sns.countplot(x="Attrition", data=df)
plt.title("Employee Attrition Distribution")
plt.show()
```



```
[7]
plt.figure(figsize=(12,8))
numeric_df = df.select_dtypes(include=np.number)
sns.heatmap(numeric_df.corr(), cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```




```
[8]
✓ 0s
plt.figure(figsize=(6,4))
sns.boxplot(x="Attrition", y="JobSatisfaction", data=df)
plt.title("Job Satisfaction vs Attrition")
plt.show()
```




```
[9]
✓ 0s
df = df.drop(["EmployeeCount","EmployeeNumber","Over18","StandardHours"], axis=1)
```

[10]
0s

```python
label_encoder = LabelEncoder()

for column in df.columns:
    if df[column].dtype == "object":
        df[column] = label_encoder.fit_transform(df[column])
```

[11]
0s

```python
X = df.drop("Attrition", axis=1)
y = df["Attrition"]
```

[12]
0s

```python
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

[13]
0s

```python
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

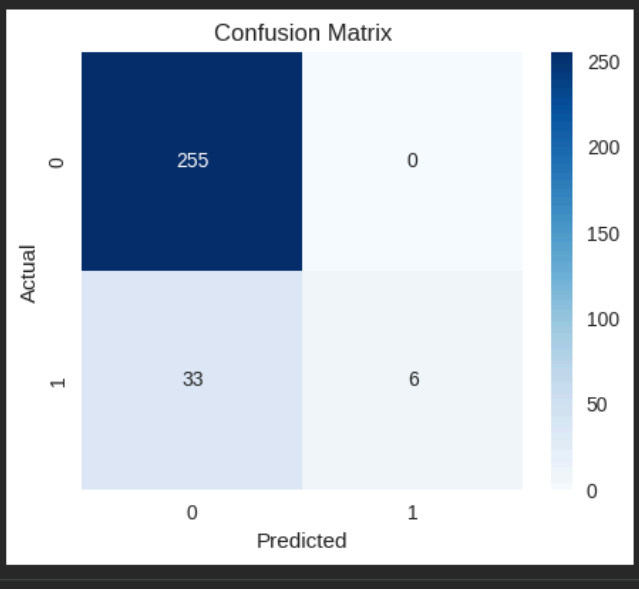
[14]
0s

```python
svm_model = SVC()

svm_model.fit(X_train, y_train)

y_pred = svm_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.8877551020408163
              precision    recall  f1-score   support

           0       0.89      1.00      0.94       255
           1       1.00      0.15      0.27        39

    accuracy                           0.89       294
   macro avg       0.94      0.58      0.60       294
weighted avg       0.90      0.89      0.85       294
```

[15]
0s

```python
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



[16]
1m

```python
param_grid = {
    "C":[0.1,1,10,100],
    "gamma":[1,0.1,0.01,0.001],
    "kernel":["rbf","linear","poly"]
}

grid = GridSearchCV(SVC(), param_grid, cv=5, verbose=2)

grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)
```

```
[CV] END .........................C=100, gamma=1, kernel=rbf; total time=   0.1s
[CV] END .........................C=100, gamma=1, kernel=rbf; total time=   0.1s
[CV] END .......................C=100, gamma=1, kernel=linear; total time=   3.1s
[CV] END .......................C=100, gamma=1, kernel=linear; total time=   5.3s
[CV] END .......................C=100, gamma=1, kernel=linear; total time=   4.3s
[CV] END .......................C=100, gamma=1, kernel=linear; total time=   2.4s
[CV] END .......................C=100, gamma=1, kernel=linear; total time=   5.7s
[CV] END .........................C=100, gamma=1, kernel=poly; total time=   0.1s
[CV] END .........................C=100, gamma=1, kernel=poly; total time=   0.1s
[CV] END .........................C=100, gamma=1, kernel=poly; total time=   0.1s
[CV] END .........................C=100, gamma=1, kernel=poly; total time=   0.1s
[CV] END .........................C=100, gamma=1, kernel=poly; total time=   0.1s
[CV] END ........................C=100, gamma=0.1, kernel=rbf; total time=   0.1s
[CV] END ........................C=100, gamma=0.1, kernel=rbf; total time=   0.1s
[CV] END ........................C=100, gamma=0.1, kernel=rbf; total time=   0.1s
[CV] END ........................C=100, gamma=0.1, kernel=rbf; total time=   0.1s
[CV] END ........................C=100, gamma=0.1, kernel=rbf; total time=   0.1s
[CV] END ......................C=100, gamma=0.1, kernel=linear; total time=   3.3s
[CV] END ......................C=100, gamma=0.1, kernel=linear; total time=   4.0s
[CV] END ......................C=100, gamma=0.1, kernel=linear; total time=   5.7s
[CV] END ......................C=100, gamma=0.1, kernel=linear; total time=   2.4s
[CV] END ......................C=100, gamma=0.1, kernel=linear; total time=   3.8s
[CV] END ........................C=100, gamma=0.1, kernel=poly; total time=   0.1s
[CV] END ........................C=100, gamma=0.1, kernel=poly; total time=   0.1s
```

```
[CV] END ....................C=100, gamma=0.001, kernel=poly; total time=   0.0s
Best Parameters: {'C': 100, 'gamma': 0.001, 'kernel': 'rbf'}
```

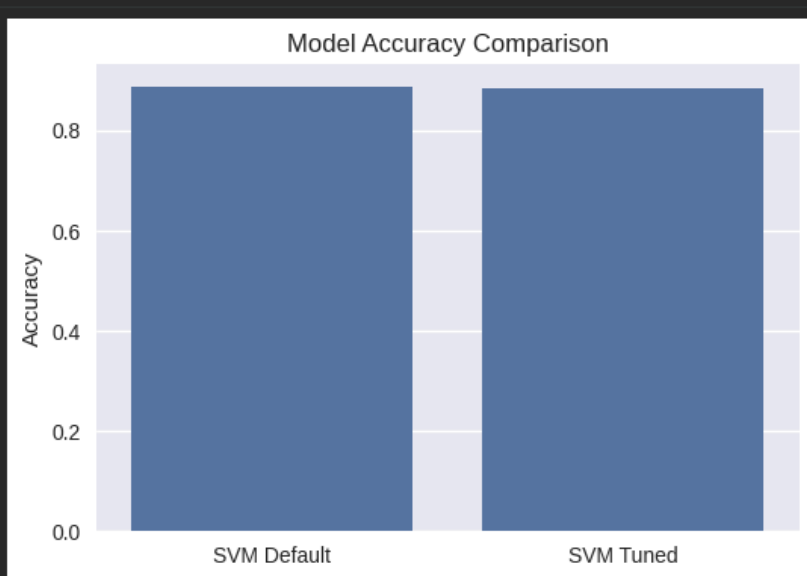
[17]
✓ 0s

```
best_svm = grid.best_estimator_

y_pred_tuned = best_svm.predict(X_test)

print("Tuned Accuracy:", accuracy_score(y_test, y_pred_tuned))
print(classification_report(y_test, y_pred_tuned))
```

```
Tuned Accuracy: 0.8843537414965986
              precision    recall  f1-score   support

           0       0.90      0.98      0.94       255
           1       0.65      0.28      0.39        39

    accuracy                           0.88       294
   macro avg       0.77      0.63      0.66       294
weighted avg       0.87      0.88      0.86       294
```

[18]
✓ 0s

```
models = ["SVM Default","SVM Tuned"]

scores = [
accuracy_score(y_test,y_pred),
accuracy_score(y_test,y_pred_tuned)
]

plt.figure(figsize=(6,4))
sns.barplot(x=models,y=scores)
plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")
plt.show()
```



Conclusion

In this experiment, Support Vector Machine (SVM) was implemented to predict employee attrition. After performing hyperparameter tuning using GridSearchCV, the optimal model used an RBF kernel with C = 100 and gamma = 0.001.

The model achieved an accuracy of approximately **88%**, demonstrating strong classification capability. However, due to class imbalance in the dataset, the model performed better in predicting employees who stayed compared to those who left.

This experiment demonstrates the effectiveness of SVM for classification problems and highlights the importance of hyperparameter tuning and handling class imbalance in real-world datasets.